

 Państwowa Akademia Nauk Stosowanych w Krakowie		Kierunek Informatyka	
Instrukcja do ćwiczeń laboratoryjnych nr:	5	Nazwa przedmiotu: Programowanie w języku Kotlin	
Temat: Jetpack Compose Navigation – programowanie reaktywne		Tryb studiów: stacjonarne	
		Czas trwanie ćw. 2x45 min	
Autor materiałów: dr Marcin Skuba			

1. Treści programowe:

Programowanie deklaratywne, tworzenie interfejsu UI – Jetpack Compose, Kotlin

2. Cel zajęć:

Celem zajęć jest zrozumienie zasad oraz poznanie składni programowania deklaratywnego na przykładzie biblioteki JetPpack Compose do tworzenia interfejsów użytkownika w języku Kotlin.

3. Materiały dydaktyczne

Single Activity Architecture

W Jetpack Compose podejście do budowania aplikacji zmieniło się diametralnie. Choć technicznie możesz używać wielu aktywności, to współczesny standard (zalecany przez Google) mówi o czymś zupełnie innym.

Podejście to nazywa się Single Activity Architecture (Architektura Jednej Aktywności).

1. XML

W starym systemie każda nowa sekcja aplikacji (Logowanie, Lista, Detale) zazwyczaj była osobną Activity. Przełączanie się między nimi było "ciężkie" dla systemu i wymagało rejestrowania każdej aktywności w pliku AndroidManifest.xml.

2. Compose

W Compose cała Twoja aplikacja zazwyczaj "żyje" w jednej głównej aktywności (np. MainActivity). Zamiast przełączać Activity, po prostu podmieniamy funkcje @Composable na ekranie.

Działa to jak w przeglądarce internetowej – adres URL się zmienia, ale strona nie przeładowuje się całkowicie, tylko podmienia treść na środku.

3. Czym nawigujemy między widokami?

Do zarządzania tymi "podmianami" używamy biblioteki Jetpack Compose Navigation. Zamiast wywoływać startActivity(), używamy obiektu NavController.

Oto trzy kluczowe elementy tej nawigacji:

- **NavController**: Twój "pilot". Pamięta historię (ekran wstecz) i wykonuje polecenie "idź do ekranu B".
- **NavHost**: To "kontener" lub "okno", w którym odbywa się podmiana. Tutaj definiujesz listę wszystkich dostępnych ekranów w aplikacji.
- **Composable Destination**: Każdy Twój ekran (np. LoginScreen(), HomeScreen()) to po prostu funkcja, do której przypisujesz unikalną nazwę (tzw. "route").

Poniższy przykład przedstawia definiowanie tras jako ciąg znaków. Dalszej części instrukcji pokazany jest lepszy sposób gdzie trasy będą opisywane przez obiekty i klasy, a przekazywanie wartości będzie bezpieczne i będzie można przekazywać obiekty.

```
val navController = rememberNavController()
```

```
NavHost(navController = navController, startDestination = "ekran_listy") {

    // Definiujemy trasę "ekran_listy" przekazując dane
    composable("ekran_listy") {
        ListaProduktowScreen(onProductClick = { id ->
            navController.navigate("detale/$id")
        })
    }
    // Definiujemy trasę "detale" z danymi
    composable("detale/{productId}") { backStackEntry ->
        val id = backStackEntry.arguments?.getString("productId")
        SzczegolyProduktuScreen(id)
    }
}
```

1. Szybkość: Podmiana funkcji **@Composable** jest niemal natychmiastowa. Nie ma ciężkiego przeładowania całego okna systemowego.
2. Płynne animacje: Możesz bardzo łatwo animować przechodzenie jednego elementu w drugi (tzw. Shared Element Transitions), ponieważ oba widoki należą do tego samego okna.
3. Współdzielenie danych: Dużo łatwiej jest przekazać dane między ekranami, gdy nie musisz ich "pakować" do Intenta (choć nadal używamy do tego ViewModeli).
4. Czysty Manifest: Nie musisz wpisywać każdego ekranu do pliku AndroidManifest.xml.

Kiedy jednak użyć nowej Activity?

Mimo wszystko, nową aktywność tworzymy bardzo rzadko, głównie w dwóch przypadkach:

- Gdy Twój projekt to "mieszanka" (część jest w starym XML, część w Compose).
- Gdy chcesz, aby jakaś część aplikacji była całkowicie odizolowana (np. osobny proces dla aparatu lub płatności).

Podsumowując: W 99% przypadków w Compose używasz jednej aktywności i biblioteki Navigation, aby dynamicznie zamieniać funkcje @Composable na ekranie.

Programowanie reaktywne

Programowanie reaktywne (**Reactive Programming**) to paradygmat programowania skoncentrowany na **strumieniach danych i propagacji zmian**.

Zamiast pisać kod, który wykonuje się krok po kroku (imperatywnie), projektujesz system, który automatycznie "reaguje" na nowe informacje, zdarzenia lub zmiany stanu.

Pozwala tworzyć aplikacje, które są **skalowalne i odporne na błędy** (tzw. *Responsive Systems*). Świetnie sprawdza się w systemach czasu rzeczywistego, interfejsach użytkownika oraz architekturze mikroserwisów. Z programowaniem reaktywnym mamy do czynienia z Jetpack Compose.

Potrzebne zależności i pluginy w pliku build.gradle.kts(:app):

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.android)
    alias(libs.plugins.kotlin.compose)
    kotlin("plugin.serialization") version "2.0.0"
}

...

dependencies {
    //...
    // Nawigacja
    implementation("androidx.navigation:navigation-compose:2.9.0")

    // Serializacja JSON (wymagana do obsługi tras)
    implementation("org.jetbrains.kotlinx:kotlinx-serialization-json:1.7.0")
}
```

PRZYKŁAD 1

Najprostszy przykład przedstawiający aktywność z dwoma widokami zarządzanymi przez kontroler **bez przekazywania wartości**.

Cały kod dwóch widoków znajduje się bezpośrednio w NavHost. Bez oddzielnych funkcji i bez przekazywania lambdy jako parametru.

```
import CounterViewModel
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.material3.Button
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.rememberNavController
import com.example.jpc_nav2.ui.theme.JPC_NAV2Theme
import kotlinx.serialization.Serializable

@Serializable
object ScreenRout1

@Serializable
object ScreenRout2

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            JPC_NAV2Theme {
                Navigarion()
            }
        }
    }
}

@Composable
fun Navigarion(){
    val navConroller = rememberNavController()
    NavHost(navController = navConroller, startDestination = ScreenRout1) {
        composable<ScreenRout1>{
            Column(
                modifier = Modifier.fillMaxSize(),
                verticalArrangement = Arrangement.Center,
                horizontalAlignment = Alignment.CenterHorizontally
            ) {
                Text(text = "Pierwszy ekran")
                Button(onClick = {
                    navConroller.navigate(ScreenRout2)
                }) {
                    Text(text = "Przejdź do drugiego ekranu")
                }
            }
        }
        composable<ScreenRout2>{
            Column(
                modifier = Modifier.fillMaxSize(),
                verticalArrangement = Arrangement.Center,
                horizontalAlignment = Alignment.CenterHorizontally
```

PRZYKŁAD 2

```
@Composable
```

```

fun AppNavigation() {
    val navController = rememberNavController()
    NavHost(
        navController = navController,
        startDestination = HomeRoute
    ){
        composable<HomeRoute>{
            var text by remember { mutableStateOf("") }

            Column(
                modifier = Modifier.fillMaxSize(),
                horizontalAlignment = Alignment.CenterHorizontally,
                verticalArrangement = Arrangement.Center
            ) {
                Text("Pierwszy ekran", style = MaterialTheme.typography.headlineMedium)
                Spacer(modifier = Modifier
                    .height(30.dp))
                OutlinedTextField(
                    value = text,
                    onValueChange = {text=it},
                    label = { Text("Wpisz wiadomość") },
                    modifier = Modifier.fillMaxWidth()
                )
                Button(onClick = {
                    navController.navigate(DetailsRoute(text))
                },
                    modifier = Modifier.padding(top = 10.dp)) {
                    Text("Wyślij")
                }
            }
        }
        composable<DetailsRoute> { backStackEntry ->
            val arg: DetailsRoute = backStackEntry.toRoute()
            Column(
                modifier = Modifier.fillMaxSize(),
                horizontalAlignment = Alignment.CenterHorizontally,
                verticalArrangement = Arrangement.Center
            ) {
                Text(arg.userText, style = MaterialTheme.typography.headlineMedium)
                Button(onClick = {
                    navController.popBackStack()
                },
                    modifier = Modifier.padding(top = 10.dp)) {
                    Text("Wróć")
                }
            }
        }
    }
}

```

1. **@Serializable** to "znacznik", który mówi programowi: "Przygotuj tę klasę tak, aby można ją było łatwo zamienić na dane tekstowe i przesłać dalej". Jest to niezbędne, aby system nawigacji mógł przekazać informacje o tym, gdzie użytkownik chce przejść.

2. **object HomeRoute**

- Reprezentuje ekran główny (Home).
- Dlaczego **object**: Używamy obiektu, bo ekran główny zazwyczaj jest tylko jeden i nie potrzebuje żadnych dodatkowych danych (argumentów), żeby się wyświetlić. Po prostu "jest".

3. **data class DetailsRoute(val userText: String)**

- Reprezentuje ekran szczegółów (Details).
- Dlaczego **data class**: Używamy klasy, ponieważ ten ekran potrzebuje konkretnych informacji, aby wiedzieć, co pokazać.

- **val userText: String:** To "bagaż", który zabierasz ze sobą na ten ekran. W tym przypadku przekazujesz tekst (np. to, co użytkownik wpisał wcześniej), aby wyświetlić go na ekranie szczegółów.

W skrócie:

- HomeRoute = "Idź do domu" (bez pytań).
- DetailsRoute = "Idź do szczegółów i weź ze sobą ten konkretny tekst".

Przekazywanie danych

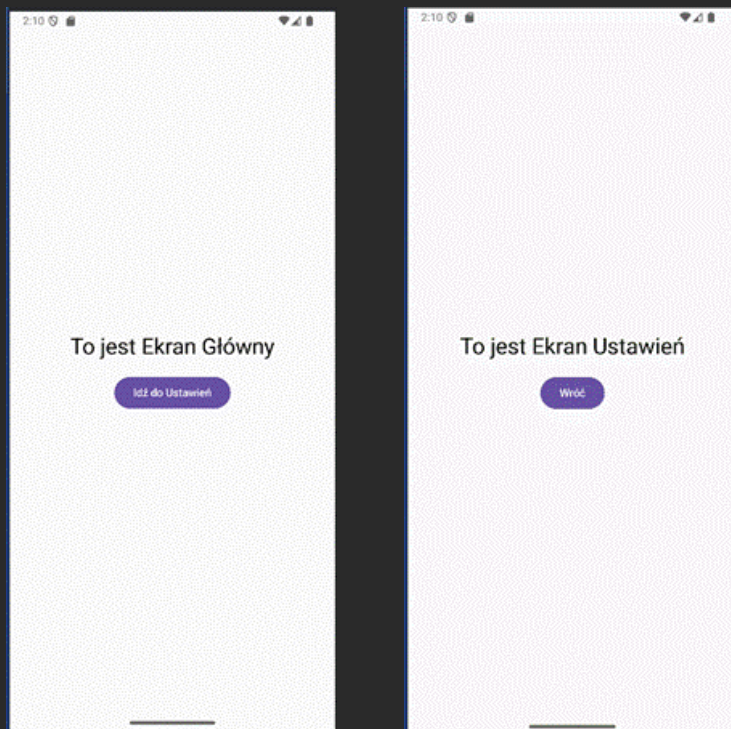
Mechanizm przedstawiony w kodzie powyżej to **Type-Safe Navigation** (Bezpieczna Typowo Nawigacja), która zastąpiła ręczne budowanie ciągów znaków (Stringów). Oto najważniejsze zasady jej działania:

- **Definiowanie tras jako obiektów:** Trasy nie są już zwykłymi napisami, lecz klasami danych (data class) lub obiektami oznaczonymi adnotacją `@Serializable`. Dzięki temu parametry (jak `userText`) są zdefiniowane jako konkretne właściwości klasy.
- **Wysyłanie danych przez instancję klasy:** Aby przejść do innego ekranu, wywołujesz funkcję `navigate()`, przekazując jej gotowy **obiekt** z danymi (np. `DetailsRoute(text)`). Biblioteka nawigacji automatycznie zajmuje się "spakowaniem" tych danych pod spodem.
- **Odbieranie danych funkcją `toRoute()`:** Na ekranie docelowym nie musisz ręcznie wyciągać wartości z Bundle po kluczach. Funkcja `backStackEntry.toRoute<DetailsRoute>()` automatycznie odtwarza cały obiekt trasy wraz z przekazanymi wartościami.
- **Bezpieczeństwo typów (Type Safety):** Cały proces jest sprawdzany przez kompilator – jeśli zmienisz nazwę pola w `DetailsRoute` lub typ danych, kompilator natychmiast wskaże błąd w miejscach nawigacji i odbioru danych, co zapobiega błędom w trakcie działania aplikacji (Runtime).

Zamiast bawić się w ręczne doklejanie tekstu do adresu URL ("`details/"` + `text`), traktujesz trasę jak zwykły obiekt Kotlinowy, który przesyłasz między ekranami.

PRZYKŁAD 3

Przykład przedstawiający aktywność z dwoma widokami zarządzanymi przez kontroler **bez przekazywania wartości**.



MainActivity.kt (przykład w jednym pliku):

```

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.height
import androidx.compose.material3.Button
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.rememberNavController
import kotlinx.serialization.Serializable

// 1. DEFINICJA TRAS (ROUTES)
// Używamy @Serializable, aby biblioteka nawigacji rozumiała te obiekty
@Serializable
object Home
@Serializable
object Settings

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            AppNavigation()
        }
    }
}

// 2. GŁÓWNY KOMPONENT NAWIGACJI
@Composable
fun AppNavigation() {
    // Kontroler nawigacji zarządza stosem ekranów
    val navController = rememberNavController()
    NavHost(
        navController = navController,
        startDestination = Home // Ustawiamy Home jako ekran startowy
    ) {
        // Definiujemy ekran Home
        composable<Home> {
            HomeScreen(
                onGoToSettings = { navController.navigate(Settings) }
            )
        }
        // Definiujemy ekran Settings
        composable<Settings> {
            SettingsScreen(
                onBack = { navController.popBackStack() }
            )
        }
    }
}

// 3. WIDOK PIERWSZY (HOME)
@Composable
fun HomeScreen(onGoToSettings: () -> Unit) {
    Column(

```

```

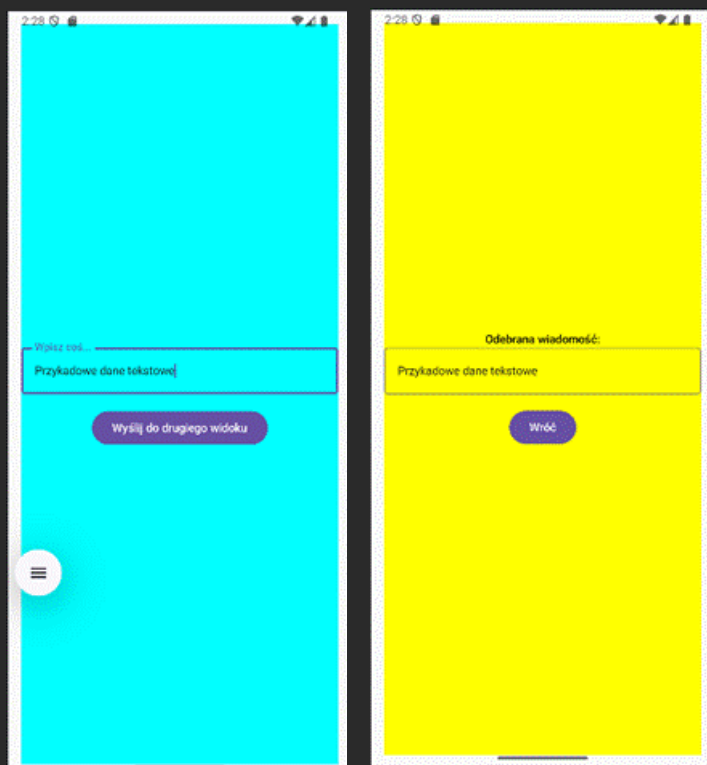
        modifier = Modifier.fillMaxSize(),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(text = "To jest Ekran Główny", style = MaterialTheme.typography.headlineMedium)
        Spacer(modifier = Modifier.height(16.dp))
        Button(onClick = onGoToSettings) {
            Text("Idź do Ustawień")
        }
    }
}

// 4. WIDOK DRUGI (SETTINGS)
@Composable
fun SettingsScreen(onBack: () -> Unit) {
    Column(
        modifier = Modifier.fillMaxSize(),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(text = "To jest Ekran Ustawień", style = MaterialTheme.typography.headlineMedium)
        Spacer(modifier = Modifier.height(16.dp))
        Button(onClick = onBack) {
            Text("Wróć")
        }
    }
}

```

PRZYKŁAD 4

Przykład przedstawiający aktywność z dwoma widokami zarządzanymi przez kontroler z **przekazywaniem wartości**. W przykładzie zdefiniowano osobne funkcje Composable z **funkcjami lambda** jako argumenty.



```

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent

```

```

import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Button
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.unit.dp
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.rememberNavController
import androidx.navigation.toRoute
import kotlinx.serialization.Serializable

// Pierwszy ekran nie potrzebuje danych wejściowych
@Serializable
object HomeRoute

// Drugi ekran potrzebuje ciągu znaków (naszej wiadomości)
@Serializable
data class DetailsRoute(val userText: String)

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            AppNavigation()
        }
    }
}

@Composable
fun AppNavigation() {
    val navController = rememberNavController()

    NavHost(
        navController = navController,
        startDestination = HomeRoute
    ) {
        // WIDOK 1: HOME
        composable<HomeRoute> {
            // Przekazujemy wpisany tekst tworząc instancję klasy DetailsRoute
            HomeScreen(onNavigateToDetails = { text ->
                navController.navigate(DetailsRoute(userText = text))
            })
        }

        // WIDOK 2: DETAILS
        composable<DetailsRoute> { backStackEntry ->

```

```
// odbiornik, który stoi na straży drugiego ekranu. Jego zadaniem jest przechwycenie
// "paczki" z danymi, rozpakowanie jej i przekazanie zawartości do widoku.
val arguments: DetailsRoute = backStackEntry.toRoute()
```

```
DetailsScreen(
    receivedText = arguments.userText,
    onBack = { navigationController.popBackStack() }
)
```

```
}
```

```
}
```

```
}
```

```
// --- WIDOK 1: EKRAN WPISYWANIA ---
```

```
@Composable
```

```
fun HomeScreen(onNavigateToDetails: (String) -> Unit) {
```

```
    // Stan przechowujący to, co wpisuje użytkownik
```

```
    var inputText by remember { mutableStateOf("") }
```

```
    Column(
        modifier = Modifier.fillMaxSize().padding(16.dp)
            .background(Color.Cyan ),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
```

```
        OutlinedTextField(
            value = inputText,
            onValueChange = { inputText = it },
            label = { Text("Wpisz coś...") },
            modifier = Modifier.fillMaxWidth()
        )
```

```
        Spacer(modifier = Modifier.height(16.dp))
```

```
        Button(
            onClick = { onNavigateToDetails(inputText) },
            enabled = inputText.isNotBlank() // Przycisk aktywny tylko gdy tekst nie jest pusty
        ) {
            Text("Wyślij do drugiego widoku")
        }
    }
}
```

```
}
```

```
// --- WIDOK 2: EKRAN ODBIORU ---
```

```
@Composable
```

```
fun DetailsScreen(receivedText: String, onBack: () -> Unit) {
```

```
    Column(
        modifier = Modifier.fillMaxSize().padding(16.dp)
            .background(Color.Yellow),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
```

```
        Text("Odebrana wiadomość:", style = MaterialTheme.typography.labelLarge)
```

```
        // Wyświetlamy odebrany tekst w polu tekstowym (tylko do odczytu)
```

```
        OutlinedTextField(
            value = receivedText,
            onValueChange = {},
            readOnly = true,
            modifier = Modifier.fillMaxWidth()
        )
```

```
        Spacer(modifier = Modifier.height(16.dp))
```

```
        Button(onClick = onBack) {
            Text("Wróć")
        }
    }
}
```

```
}
```

```
}
```

Na czym polega ten mechanizm przekazywania funkcji lambda z parametrem

```
fun HomeScreen(onNavigateToDetails: (String) -> Unit)
```

Przekazywanie `onNavigateToDetails: (String) -> Unit` to wzorec State Hoisting (wynoszenie stanu/zdarzeń w górę). Zamiast pozwalać ekranowi HomeScreen samodzielnie decydować o nawigacji, ekran ten jedynie "zgłasza" zdarzenie: *"Użytkownik kliknął przycisk i chce przekazać ten tekst"*.

Kluczowe zalety

- 1. **Rozdzielenie odpowiedzialności** (Separation of Concerns): HomeScreen odpowiada tylko za wyświetlanie interfejsu, a nie za logikę poruszania się po całej aplikacji.
- 2. **Testowalność**: Możesz łatwo przetestować HomeScreen, sprawdzając tylko, czy funkcja `onNavigateToDetails` została wywołana z odpowiednim tekstem, bez uruchamiania całego mechanizmu nawigacji.
- 3. **Wielorazowość**: Ten sam ekran mógłby w innym miejscu aplikacji wykonywać inną akcję po kliknięciu (np. zapisywać tekst do bazy zamiast nawigować).

Jak to działa

- 1. Definicja: W HomeScreen definiujesz "slot" na funkcję (callback).
- 2. Wywołanie: Wewnątrz kompozowalnego elementu (np. w `Button(onClick = { onNavigateToDetails(text) })`) uruchamiasz tę funkcję, przekazując dane.
- 3. Realizacja: Dopiero w miejscu, gdzie wywołujesz HomeScreen (zazwyczaj w NavHost), określasz, co faktycznie ma się stać.

```
HomeScreen(onNavigateToDetails = { text ->
    navController.navigate(DetailsRoute(userText = text))
```

Wyjaśnienie zapisu `onNavigateToDetails = { text -> ... }`

Ten zapis to implementacja przekazana do ekranu:

- Lambda jako łącznik: Jest to funkcja anonimowa (lambda), która "nasłuchuje" na sygnał z wnętrza HomeScreen.
- Przechwycenie parametru: Zmienna text (po lewej stronie strzałki ->) to dokładnie ta wartość, którą HomeScreen wysłał wewnątrz swojej logiki.
- Wykonanie nawigacji: `navController.navigate(...)` to konkretna reakcja na zdarzenie. Tutaj następuje mapowanie surowego tekstu (String) na bezpieczny typowo obiekt trasy (DetailsRoute), co uruchamia proces przejścia do nowego widoku.

Traktujcie to jak pilot do telewizora. Przycisk na pilocie (HomeScreen) nie wie, jak działa elektronika w środku – on tylko wysyła sygnał. To telewizor (NavHost) wie, że odebranie sygnału "3" oznacza przełączenie kanału.

PRZYKŁAD 4.1

Poniższy przykład przedstawia deklarację oraz wywołanie funkcji z parametrem funkcji lambda **dla trzech parametrów**:

Najpierw upewniamy się, że klasa `DetailsRoute` jest przygotowana na przyjęcie trzech wartości:

```
@Serializable
data class DetailsRoute(
    val name: String,
    val surname: String,
    val age: Int
)
```

Zmieniamy sygnaturę `onNavigateToDetails`, aby przyjmowała trzy parametry: (String, String, Int).

```

@Composable
fun HomeScreen(onNavigateToDetails: (String, String, Int) -> Unit) {
    // Przykładowe dane (w realnej aplikacji pochodziłyby z pól TextField)
    val name = "Jan"
    val surname = "Kowalski"
    val age = 25

    Button(onClick = {
        // Wywołujemy funkcję z trzema argumentami
        onNavigateToDetails(name, surname, age)
    }) {
        Text("Wyślij komplet danych")
    }
}

```

W miejscu, gdzie wywołujemy **HomeScreen**, odbieramy te trzy parametry w lambdzie i przekazujemy je do klasy trasy:

```

HomeScreen (onNavigateToDetails = { n, s, a ->
    // Tworzymy obiekt DetailsRoute korzystając z odebranych wartości
    navController.navigate(DetailsRoute(name = n, surname = s, age = a))
})

```

ViewModel:

Co to jest ViewModel i do czego służy?

ViewModel to klasa, która przechowuje i zarządza danymi związanymi z interfejsem użytkownika (UI). Jest częścią architektury zalecanej przez Google.

Główne zadania:

- **Przetwarzanie zmian konfiguracji:** Najważniejsza cecha – ViewModel nie "umiera", gdy użytkownik obróci ekran (w przeciwieństwie do Activity). Dane nie znikają.
- **Oddzielenie logiki od widoku:** Widok (Compose) ma tylko wyświetlać dane, a ViewModel ma decydować, jak te dane zmieniać.
- **Zarządzanie stanem:** To "mózg" ekranu, który trzyma aktualne informacje (np. czy przycisk jest aktywny, co wpisano w pole tekstowe).

Wymagane zależności:

```

dependencies {
    // 1. Podstawowa biblioteka ViewModel (logika i klasa ViewModel)
    implementation("androidx.lifecycle:lifecycle-viewmodel-ktx:2.8.0")

    // 2. Integracja z Compose (pozwala używać funkcji viewModel() w @Composable)
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.0")
}

```

Przykład: Licznik punktów (Counter)

W tym przykładzie **ViewModel** trzyma stan licznika, a widok go tylko wyświetla.

1. Klasa ViewModel

Musimy dziedziczyć po klasie ViewModel(). Używamy MutableState, aby Compose wiedział, kiedy odświeżyć ekran.

```

import androidx.compose.runtime.mutableStateOf
import androidx.lifecycle.ViewModel

class CounterViewModel : ViewModel() {
    // Stan przechowywany w ViewModelu (prywatny, by nikt z zewnątrz go nie zepsuł)
    var count = mutableStateOf(0)
}

```

```
private set
```

```
// Funkcja zmieniająca stan (logika biznesowa)
```

```
fun incrementCount() {
```

```
    count.value++
```

```
}
```

```
}
```

2. Widok (UI) w Compose

Widok "obserwuje" ViewModel i wywołuje jego funkcje.

```
@Composable
```

```
fun CounterScreen(viewModel: CounterViewModel = viewModel()) {
```

```
    Column(
```

```
        modifier = Modifier.fillMaxSize(),
```

```
        verticalArrangement = Arrangement.Center,
```

```
        horizontalAlignment = Alignment.CenterHorizontally
```

```
) {
```

```
    Text(
```

```
        text = "Liczba kliknięć: ${viewModel.count.value}",
```

```
        style = MaterialTheme.typography.headlineMedium
```

```
)
```

```
    Button(onClick = { viewModel.incrementCount() }) {
```

```
        Text("Kliknij mnie!")
```

```
    }
```

```
}
```

```
}
```

Dlaczego to działa tak dobrze? (Opis mechanizmu)

1. Stabilność danych: Jeśli wpiszesz coś w **OutlinedTextField** lub nabijesz punkty w liczniku, a następnie obrócisz telefon, **CounterScreen** zostanie narysowany od nowa, ale **CounterViewModel** pozostanie ten sam. Dane zostaną natychmiast przywrócone.
2. Jedno źródło prawdy (Single Source of Truth): Tylko **ViewModel** ma uprawnienia do zmiany zmiennej **count**. Dzięki temu unikasz błędów, w których kilka elementów interfejsu próbuje sprzecznie zmieniać tę samą wartość.
3. Czysty kod: Twoje funkcje **@Composable** stają się krótsze i czytelniejsze, bo cała "matematyka" i operacje na danych lądują w osobnej klasie Kotlinowej.

PRZYKŁAD 5

Przykład przedstawiający aktywność z dwoma widokami zarządzanymi przez kontroler z przekazywaniem wartości przez **ViewModel**:

Plik **DetailsViewModel.kt**:

```
package com.example.jpc_nav
```

```
import androidx.lifecycle.SavedStateHandle
```

```
import androidx.lifecycle.ViewModel
```

```
import androidx.navigation.toRoute
```

```
class DetailsViewModel(savedStateHandle: SavedStateHandle) : ViewModel() {
```

```
    // Automatycznie wyciągamy dane z trasy DetailsRoute
```

```
    // To jest bezpieczne typowo (Type-Safe)!
```

```
    private val route = savedStateHandle.toRoute<DetailsRoute>()
```

```
    val receivedText: String = route.userText
```

```
}
```

Plik MainActivity.kt:

```
package com.example.jpc_nav

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Button
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.rememberNavController
import kotlinx.serialization.Serializable

// Pierwszy ekran nie potrzebuje danych wejściowych
@Serializable
object HomeRoute

// Drugi ekran potrzebuje ciągu znaków (naszej wiadomości)
@Serializable
data class DetailsRoute(val userText: String)

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            AppNavigation()
        }
    }
}

@Composable
fun AppNavigation() {
    val navController = rememberNavController()

    NavHost(navController = navController, startDestination = HomeRoute) {
        composable<HomeRoute> {
            HomeScreen(onNavigate = { text ->
                navController.navigate(DetailsRoute(text))
            })
        }

        composable<DetailsRoute> {
            // Inicjalizujemy ViewModel. On sam sobie poradzi z pobraniem danych z trasy.
            val viewModel: DetailsViewModel = viewModel()

            DetailsScreen(
```

```
textFromVm = viewModel.receivedText,  
onBack = { navController.popBackStack() }  
)  
}  
}
```

```
@Composable  
fun HomeScreen(onNavigate: (String) -> Unit) {  
    var text by remember { mutableStateOf("") }  
    Column(  
        modifier = Modifier.fillMaxSize().padding(16.dp),  
        horizontalAlignment = Alignment.CenterHorizontally,  
        verticalArrangement = Arrangement.Center)  
    {  
        OutlinedTextField(  
            value = text,  
            onValueChange = { text = it },  
            label = { Text("Wpisz wiadomość") },  
            modifier = Modifier.fillMaxWidth()  
        )  
        Button(onClick = { onNavigate(text) }, modifier = Modifier.padding(top = 8.dp)) {  
            Text("Wyślij")  
        }  
    }  
}
```

```
@Composable  
fun DetailsScreen(textFromVm: String, onBack: () -> Unit) {  
    Column(modifier = Modifier.fillMaxSize().padding(16.dp),  
        horizontalAlignment = Alignment.CenterHorizontally,  
        verticalArrangement = Arrangement.Center)  
    {  
        Text("Dane z ViewModelu:", style = MaterialTheme.typography.labelSmall)  
        Text(text = textFromVm, style = MaterialTheme.typography.headlineMedium)  
        Button(onClick = onBack, modifier = Modifier.padding(top = 16.dp)) {  
            Text("Wróć")  
        }  
    }  
}
```

Zasoby (kolory, czcionki, kształty):

ui.theme - folder generowany automatycznie przy tworzeniu nowego projektu w Jetpack Compose. Jest to miejsce, w którym definiujesz "DNA" swojej aplikacji: kolory, czcionki i kształty.

Zamiast wpisywać **Color.Red** czy 16.sp bezpośrednio w widoku (tzw. hardkodowanie), używasz **motywu**, aby Twoja aplikacja była spójna i łatwo obsługiwała np. **tryb ciemny**.

Zawartość folderu **ui.theme**:

- **Color.kt**: Tu definiujesz konkretne wartości kolorów (np. `val Purple80 = Color(0xFFD0BCFF)`).
- **Type.kt**: Tu ustawiasz style tekstu (typografia).
- **Theme.kt**: To "serce" motywu. Tu decydujesz, jakie kolory przypisać do trybu jasnego i ciemnego.
- **Shape.kt**: (Opcjonalnie) Definicje zaokrągleń rogów.

Przykład odwołania do zasobów:

```
@Composable  
fun StyledComponent() {  
    Column(  
        modifier = Modifier.fillMaxSize().padding(16.dp),  
        horizontalAlignment = Alignment.CenterHorizontally,  
        verticalArrangement = Arrangement.Center)  
    {  
        Text("Hello World", style = MaterialTheme.typography.headlineMedium, color = Color.Red)  
    }  
}
```

```

modifier = Modifier
    .background(MaterialTheme.colorScheme.background) // Używa koloru tła z
motywu
    .padding(16.dp)
) {
    Text(
        text = "Nagłówek aplikacji",
        // Używa stylu zdefiniowanego w Theme.kt
        style = MaterialTheme.typography.headlineMedium,
        // Używa koloru podstawowego z Theme.kt
        color = MaterialTheme.colorScheme.primary
    )

    Text(
        text = "To jest opis używający koloru 'onSurface'.",
        style = MaterialTheme.typography.bodySmall,
        color = MaterialTheme.colorScheme.onSurface
    )
    Text(
        text = "Tekst z kolorem pobranym bezpośrednio z pliku Color.kt",
        color = MojKolor // pobrany z pliku Color.kt
    )
}
}

```

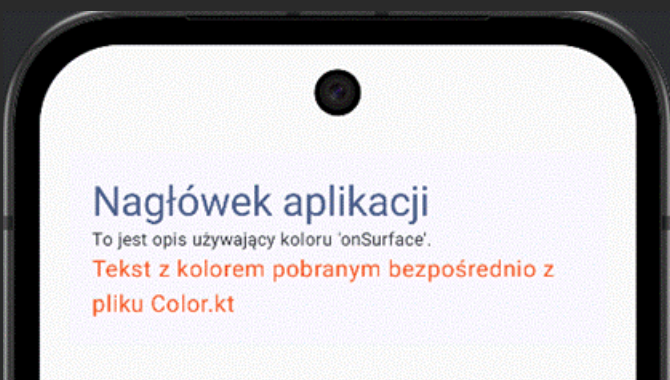
Plik Color.kt

```

package com.example.mpje_jpc3.ui.theme
import androidx.compose.ui.graphics.Color

val Purple80 = Color(0xFFD0BCFF)
val PurpleGrey80 = Color(0xFFCCC2DC)
val Pink80 = Color(0xFFE8BFD0)
val Purple40 = Color(0xFF6650a4)
val PurpleGrey40 = Color(0xFF625b71)
val Pink40 = Color(0xFF7D5260)
val MojKolor = Color(0xFFFF5722) // własny kolor

```



Zalety stosowania motywów:

- 1. Tryb Ciemny (Dark Mode):** Jeśli użyjesz `MaterialTheme.colorScheme.primary`, to gdy użytkownik przełączy telefon w tryb ciemny, Compose automatycznie podmieni jasny fiolet na ciemniejszy (zgodnie z definicją w Theme.kt). Gdybyś wpisał `Color.Blue`, zawsze zostałby niebieski.
- 2. Dynamic Color (Android 12+):** Nowoczesne telefony potrafią dopasować kolory aplikacji do tapety użytkownika. Jeśli korzystasz z zasobów motywu, Twoja aplikacja "magicznie" dostosuje się do systemu.

3. **Jedno miejsce zmian:** Jeśli klient powie: "Zmieńmy odcień zielonego w całej aplikacji", robisz to w jednym pliku (Color.kt), a nie w 50 różnych widokach.

Jak czytać nazwy w colorScheme?

System Material 3 używa logicznych nazw zamiast opisowych:

- **primary:** Główny kolor aplikacji (np. dla przycisków).
- **onPrimary:** Kolor tekstu/ikon leżących **na** kolorze primary (np. biały tekst na fioletowym przycisku).
- **surface:** Kolor powierzchni (karty, tła menu).
- **error:** Kolor dla komunikatów o błędach.

Jeśli chcesz sprawdzić, jakie kolory masz aktualnie dostępne, wpisz w kodzie `MaterialTheme.colorScheme`. i zobacz listę podpowiedzi. To samo dotyczy `MaterialTheme.typography`. (czcionki).

Zadania

Zadanie 1:

Napisz aplikację składającą się z jednej aktywności w której możliwe będzie nawigowanie między dwoma widokami. W pierwszym widoku utwórz interfejs pozwalający na wprowadzenie dwóch danych o studencie: kierunek oraz rok. Po naciśnięciu przycisku dane powinny się pojawić w drugim widoku w polu tekstowym. Umieszczony przycisk w drugim widoku powinien umożliwić powrót do pierwszej aktywności.

Dwa widoki oraz mechanizm nawigacji powinny znajdować się w jednej funkcji Composable. Do opisu tras użyj obiektów i klas. Nie używaj nazw tras w postaci tekstowej.

Zadanie 2:

Zaprojektuj i napisz aplikację składającą się z jednej aktywności w której możliwe będzie nawigowanie między trzema widokami bez przekazywania wartości. Z pierwszego widoku przechodzimy do drugiego, z drugiego wracamy do pierwszego lub do trzeciego, z trzeciego wracamy zawsze do pierwszego.

Użyj konstrukcji, w której widoki zapisane są w oddzielnych plikach w oddzielnych funkcjach.

Do opisu tras użyj obiektów i klas. Nie używaj nazw tras w postaci tekstowej.

Zadanie 3:

Zaprojektuj i napisz aplikację składającą się z jednej aktywności, w której możliwe będzie nawigowanie między trzema widokami w których należy przekazać co najmniej dwie zmienne do jednego widoku.

Użyj konstrukcji, w której widoki zapisane są w oddzielnych plikach w oddzielnych funkcjach. Użyj funkcji lambda jako argumenty funkcji Composable.

Do opisu tras użyj obiektów i klas. Nie używaj nazw tras w postaci tekstowej.